TOBB UNIVERSITY OF ECONOMICS AND TECHNOLOGY
DEPARTMENT OF COMPUTER ENGINEERING

GUIDE

(Guided User Itinerary & Destination Explorer)

Test Plan Report

QUAD-CORE Team

Ebrar Sude Doğan
Erdem Baran
Kayrahan Toprak Tosun
Tuna Kodal

Contents

1 Introduction **3**

2 Project Scope **3**
2.1 Project Title . . . 3
2.2 Brief Description of the System . . . 3
2.3 Main Functionalities of the System . . . 4

3 Features to Be Tested **4**
3.1 Feature 1: User Input Consistency and Constraint Compliance . . . 5
3.2 Feature 2: POI Pool Retrieval and Sufficiency Handling . . . 5
3.3 Feature 3: Monte Carlo Candidate Generation . . . 5
3.4 Feature 4: Scoring Function Computation Integrity . . . 5
3.5 Feature 5: Daily Selection and POI Elimination Across Days . . . 5
3.6 Feature 6: Route Computation via OSRM . . . 5
3.7 Feature 7: Interactive Replanning . . . 6
3.8 Feature 8: POI Content Delivery and Visualization . . . 6

4 Testing Methodology **6**
4.1 Unit Testing . . . 6
4.1.1 Validation Layer Tests . . . 6
4.1.2 POI Selection and Filtering Tests . . . 7
4.1.3 Monte Carlo Planning Tests . . . 7
4.1.4 Scoring Strategy Tests . . . 8
4.1.5 Replanning Logic Tests . . . 9
4.1.6 OSRM Client Interaction Tests . . . 9
4.1.7 Route Assembly Tests . . . 10
4.1.8 Content Aggregation Tests . . . 10
4.1.9 DTO Integrity Tests . . . 11
4.2 Integration Testing . . . 11
4.2.1 Presentation – API Boundary Integration . . . 12
4.2.2 API Boundary – Core Services Integration . . . 13
4.2.3 Core Services – Data Access Integration . . . 14
4.2.4 Core Services – Routing Integration (OSRM) . . . 14
4.2.5 Core Services Internal Integration . . . 15
4.2.6 Replanning Flow Integration . . . 16
4.2.7 Content Delivery Integration . . . 16
4.3 System Testing . . . 17
4.3.1 Scenario 1 – Medium POI Pool Planning . . . 17
4.3.2 Scenario 2 – Small POI Pool Handling . . . 18
4.3.3 Scenario 3 – Large POI Pool Upper-Bound Planning . . . 18

4.3.4 Scenario 4 – Replanning with Structural Edits 19
4.3.5 Scenario 5 – Replanning with POI Addition 20
4.3.6 Scenario 6 – Full End-to-End Workflow Validation 20
4.3.7 Scenario 7 – Data Source Transparency Accessibility Validation 21
4.4 Performance Testing 21
4.4.1 Performance Objective 21
4.4.2 Latency Evaluation Under Typical Usage 22
4.4.3 Upper-Bound Scenario Analysis 22
4.4.4 Replanning Performance Considerations 23
4.4.5 Resource Usage Evaluation Plan 24
4.5 User Testing 24
4.5.1 Target Participants 25
4.5.2 Test Procedure 25
4.5.3 User Test Scenarios 25
4.5.4 Evaluation Method and Questionnaire 26
4.5.5 Questionnaire Items 26
4.5.6 Success Criteria and Analysis Plan 27

5 Test Environment 28
5.1 Hardware 28
5.2 Software 29

6 Test Cases 29

7 Risks and Limitations of the Test Strategy 36
7.1 Heuristic Optimality Not Formally Verified 36
7.2 Dataset Quality Not Exhaustively Audited 36
7.3 External Routing Accuracy Outside Test Scope 37
7.4 Replanning Preference Deviation 37
7.5 Performance Under Extreme Concurrency 37
7.6 Subjective Evaluation Variability 37

8 Conclusion 38

A Requirement Traceability Matrix 38

References 39

1 Introduction

This Test Plan defines the verification and validation strategy for the GUIDE (Guided User Itinerary & Destination Explorer) system. The purpose of this document is to explain how the implemented functionalities will be tested in alignment with the requirements specified in the Project Specification Report and the architectural decisions documented in the High-Level and Low-Level Design reports [1, 2, 3]. The testing approach is structured to ensure consistency between requirements, layered architectural components, and their concrete implementations. Through this alignment, the plan establishes a systematic and traceable evaluation process before final submission.

2 Project Scope

2.1 Project Title

GUIDE – Guided **U**ser **I**tinerary **D**estination **E**xplorer

2.2 Brief Description of the System

GUIDE is a web-based travel planning and destination exploration system designed to transform user-defined constraints into structured, route-aware, multi-day itineraries. Instead of relying on a manual trip composition across multiple platforms, GUIDE provides an integrated environment that combines itinerary construction, route computation, and contextual content delivery within a single workflow.

Users provide constrained inputs such as destination city, trip duration, target daily distance range, and selected POI categories. Based on these inputs, the system retrieves a pool of candidate Points of Interest (POIs) filtered by geographic and categorical criteria.

If the number of available POIs is insufficient for the requested trip duration, the system provides corrective feedback, such as suggesting an increase in category selection or an adjustment in the number of trip days. If sufficient POIs are available, itinerary construction proceeds using a Monte Carlo–based candidate generation strategy.

For each day of the trip, multiple randomized POI subsets within defined size bounds are generated from the available POI pool. Each candidate subset is evaluated using a multi-criteria scoring function that considers the average popularity score, category diversity, and proximity to the target distance constraint (using straight-line approximation), spatial continuity between consecutive POIs, and the number of POIs included. The highest-scoring candidate is selected for that day, and its POIs are removed from the pool before constructing subsequent days.

Once daily POI sets are finalized, travel routes are computed using a locally deployed OSRM instance based on OpenStreetMap data. Distance, duration, and geometry outputs are then used for map-based visualization.

The system operates in a stateless manner at the service layer. All planning and replanning operations are performed using explicitly provided request data, without maintaining server-side session state. In addition to route planning, GUIDE enriches the exploration experience by providing curated textual descriptions, image references, and optional multilingual audio

guidance for selected POIs.

The scope of this project includes itinerary generation, routing integration, content delivery, and interactive replanning within the geographic boundary of Türkiye. Real-time traffic integration, dynamic POI operational updates, and persistent user accounts are intentionally excluded from the current scope.

2.3 Main Functionalities of the System

The primary functional capabilities of GUIDE are listed below:

- **Preference-Based Multi-Day Itinerary Generation** The system generates structured, multi-day travel itineraries based on user-defined constraints such as destination city, trip duration, selected categories, and daily distance preferences.
- **Algorithmic POI Selection and Scoring** GUIDE constructs daily POI subsets using a Monte Carlo–based candidate generation strategy combined with a configurable multi-criteria scoring function that balances popularity, category diversity, distance proximity, spatial continuity, and POI count.
- **Route Computation via OSRM Integration** For each generated itinerary, the system computes travel routes between selected POIs using a locally deployed OSRM routing engine, and provides distance, duration, and geometry data for navigation and visualization.
- **Interactive Replanning Capability** Users can iteratively refine generated itineraries by adding, removing, or reordering specific POIs. The system processes replanning requests and recomputes affected route segments accordingly.
- **POI Detail Exploration and Content Delivery** The system provides structured POI information, including curated textual descriptions, visual media references, and optional multilingual audio guidance.
- **Interactive Map-Based Visualization** Generated itineraries and route geometries are presented through an interactive map interface that supports itinerary highlighting, zooming, and spatial exploration.
- **End-to-End Planning Workflow Orchestration** GUIDE integrates itinerary generation, candidate evaluation, routing computation, and content presentation into a unified workflow, enabling users to move seamlessly from preference input to structured exploration output within a single platform.

3 Features to Be Tested

Testing activities in this project focus on validating the functional capabilities and algorithmic workflow of the GUIDE system. The following feature groups define the scope of verification.

3.1 Feature 1: User Input Consistency and Constraint Compliance

Since the frontend restricts users to predefined ranges and selectable options, testing focuses on the consistency of the normalized request structure rather than handling arbitrary invalid input. Verification ensures that the selected city, trip duration, distance range, and category filters are represented correctly in backend requests and propagated consistently across planning and replanning flows.

In addition, the generated outputs are validated against the original input constraints. This includes confirming that the number of planned days reflects the requested duration after feasibility-aware adjustment where necessary, selected POIs belong to the specified city and categories, and route characteristics remain within the defined distance preferences.

3.2 Feature 2: POI Pool Retrieval and Sufficiency Handling

This feature validates POI pool formation based on city and category filters. Testing includes correct POI filtering behavior and the handling of insufficient POI availability, where the system provides corrective guidance, such as suggesting additional categories or an adjusted day count.

3.3 Feature 3: Monte Carlo Candidate Generation

This feature covers randomized candidate subset generation from the POI pool. Testing verifies candidate count, subset-size bounds, absence of duplicates within a candidate set, and repeatability characteristics under controlled randomness (e.g., fixed seed for test runs).

3.4 Feature 4: Scoring Function Computation Integrity

This feature validates the correctness of score computation mechanics rather than subjective "ground-truth" ranking. Testing evaluates score stability and expected properties, including:

(i) monotonic behavior when a single parameter is improved while others remain fixed,
(ii) sensitivity to weight changes (predictable shifts in score contributions),
(iii) boundedness and numerical stability (no overflow/NaN, consistent ranges),
(iv) tie-handling determinism where applicable.

The objective is to ensure the scoring implementation behaves consistently and reflects intended parameter effects, without claiming an absolute notion of "best" itinerary.

3.5 Feature 5: Daily Selection and POI Elimination Across Days

This feature verifies day-by-day itinerary construction logic. Testing ensures that the highest-scoring candidate is selected for each day, that selected POIs are removed from the pool before constructing subsequent days, and that the final itinerary contains no duplicate POIs across different days.

3.6 Feature 6: Route Computation via OSRM

This feature validates integration with the locally deployed OSRM routing engine. Testing verifies that the routing outputs are consistent with the POIs supplied to OSRM, and that distance and geometry data are returned in a stable format suitable for map rendering.

3.7 Feature 7: Interactive Replanning

This feature covers user-driven modifications (adding, removing, reordering). Testing verifies that replanning recomputes affected outputs correctly, preserves itinerary structure, prevents duplication/loss of POIs, and maintains consistency between the updated itinerary and routing results.

3.8 Feature 8: POI Content Delivery and Visualization

This feature includes retrieval of POI descriptions, image references, and optional multilingual audio assets. Testing evaluates response structure, graceful handling of partial media availability, and compatibility between backend outputs and frontend visualization components.

4 Testing Methodology

This section defines the structured testing approach adopted for the GUIDE system. The testing methodology is designed to validate the system at multiple levels of abstraction, reflecting its layered and service-oriented architecture. Since GUIDE integrates many different components, a single testing strategy is not sufficient to ensure correctness and reliability.

Unless otherwise specified, automated backend tests in this project are implemented using the Python-based testing framework pytest [4].

4.1 Unit Testing

Unit testing focuses on validating individual functions and modules in strict isolation from the rest of the system. The primary objective is to verify that algorithmic components, utility functions, and core logic units produce correct and stable outputs for controlled inputs without relying on real external dependencies.

To achieve full isolation and determinism, all external collaborators (such as routing clients, repositories, data sources, or other services) are replaced with mocks or in-memory test doubles.

4.1.1 Validation Layer Tests

Related Classes: *RequestValidator*, *ValidationResult*

This test group verifies the correctness of API-level request validation logic before business services are executed. Proper validation ensures that only structurally valid and constraint-compliant requests reach the planning and routing components.

Unit tests in this group focus on deterministic validation behavior of the following methods:

- *validateRouteRequest(RouteRequest request)*
- *validateReplanRequest(ReplanRequest request)*
- *validatePoiQuery(PoiQuery query)*
- *validateTripDaySuggestionRequest(TripDaySuggestionRequest request)*

The following aspects are verified:

- Enforcement of upper bounds such as *maxTripDays*, *maxCategoryCount*, and *maxPoisPerDay*.
- Validation of minimum daily distance constraints.
- Detection of missing or null mandatory fields.
- Consistency validation of replanning payloads (e.g., ensuring that referenced POIs exist in the provided itinerary).
- Correct construction of *ValidationResult* objects (valid flag, error messages, and warnings).

All validation tests are implemented as deterministic input - output assertions without external dependencies.

4.1.2 POI Selection and Filtering Tests

Related Classes: *PoiService*, *PoiRepository*

This test group verifies the correctness of candidate POI retrieval and constraint-based filtering logic before itinerary construction. Since itinerary quality directly depends on the correctness of the candidate pool, deterministic filtering behavior is critical.

Unit tests focus on the following methods:

- *getCandidatePois(TravelPreferences prefs)*
- *filterByConstraints(List pois, TravelConstraints constraints)*
- *countAvailablePois(String city, List categories)*

The following aspects are verified:

- Correct filtering by city and category selection.
- Accurate counting of available POIs for feasibility checks.
- Stable behavior when the dataset is empty or insufficient.

To ensure isolation at the unit level, repository access is replaced with controlled in-memory datasets or mock data sources. This allows deterministic verification of filtering behavior without relying on external storage mechanisms.

4.1.3 Monte Carlo Planning Tests

Related Classes: *MonteCarloItineraryPlanner*

This test group verifies the correctness and structural integrity of the Monte Carlo–based candidate generation and selection mechanism used for itinerary construction. Since this component introduces controlled randomness and directly produces multi-day itinerary candidates, unit testing focuses on reproducibility, constraint compliance, structural validity, and selection consistency rather than subjective route quality.

Unit tests focus on the following methods:

- *generateCandidates(List pois, TravelConstraints constraints)*
- *selectBestFromCandidates(List candidates, TravelConstraints constraints)*

The following aspects are verified:

- The number of generated candidates respects the configured iteration limit.
- Each candidate itinerary complies with per-day POI count constraints.
- The final number of planned days remains consistent with the requested duration under the feasible planning outcome.
- No duplicate POIs appear within the same candidate itinerary, including across different days.
- All input POIs are either included in the generated itinerary structure or correctly excluded according to constraints.
- Selected best itinerary corresponds to the highest computed score.
- Deterministic behavior is achieved when a fixed random seed is provided.
- Graceful handling of edge cases such as empty POI pools, single-day trips, or insufficient feasible candidates.

To ensure deterministic verification, unit tests utilize a fixed random seed and controlled input datasets. Ranking strategy behavior is isolated to prevent interference from external dependencies.

4.1.4 Scoring Strategy Tests

Related Classes: *HeuristicPlanRanker*, *PlanRanker*

This test group verifies the correctness of the heuristic scoring mechanism used to compare candidate itineraries. Since itinerary quality cannot be evaluated against an objective ground-truth route, unit tests focus on the internal consistency and expected influence of scoring parameters.

Unit tests target the method:

- *score(Itinerary candidate, TravelPreferences prefs, TravelConstraints constraints)*

The following aspects are verified:

- Deterministic behavior: identical inputs always produce identical scores.
- Correct contribution of individual scoring components (e.g., popularity, category diversity, distance proximity).
- Expected score change when a single factor is improved while others remain constant.

- Candidate itineraries composed of the same POI set, regardless of ordering, produce identical scores when order is not a scoring factor.
- Score computation remains stable and bounded under valid candidate structures.

These tests ensure that the scoring implementation behaves consistently and reflects the intended weighting of evaluation criteria without claiming an absolute definition of route optimality.

4.1.5 Replanning Logic Tests

Related Classes: *ItineraryService*, *UserEdits*

This test group verifies the deterministic behavior of itinerary updates performed after user modifications. Replanning must produce consistent and predictable outputs based solely on the provided itinerary snapshot and user edit payload.

Unit tests target the method:

- *replan(Itinerary existing, UserEdits edits, TravelConstraints constraints)*

The following aspects are verified:

- Removal operations correctly eliminate specified POIs from the itinerary.
- Reorder operations correctly update the intra-day POI sequence.
- No unintended POI duplication or loss occurs after applying edits.
- Identical inputs produce identical replanning outputs (deterministic behavior).

These tests ensure that user edits are applied precisely to the existing itinerary, modifying only the intended elements while preserving the structure and integrity of unaffected days.

4.1.6 OSRM Client Interaction Tests

Related Classes: *OsrmClient*, *OsrmRouteResponse*

This test group verifies the correctness of request construction, response parsing, and normalized data representation at the OSRM integration boundary. The objective is not to validate routing optimality, but to ensure that routing responses are consistently translated into internal structures.

Unit tests target the following methods:

- *route(List waypoints, RoutingProfile profile)*
- *trip(List waypoints, RoutingProfile profile)*

The following aspects are verified:

- Correct construction of OSRM endpoint URLs.
- Proper serialization of waypoint coordinates.

- Accurate extraction of distance, duration, and geometry fields.
- Proper instantiation of *OsrmRouteResponse*.
- Stable behavior when optional or incomplete fields are returned.
- Appropriate exception handling for malformed responses.

External HTTP communication is replaced with mocked responses to ensure deterministic and isolated verification of parsing and normalization logic.

4.1.7 Route Assembly Tests

Related Classes: *RouteAssembler*

This test group verifies the correctness of the route assembly logic that transforms normalized routing responses into internal RoutePlan and RouteSegment structures. The objective is to ensure that routing outputs are consistently mapped to the corresponding itinerary days.

Unit tests target the following method:

- *assemble(Itinerary itinerary, List responses)*

The following aspects are verified:

- Each itinerary day is matched with the correct routing response.
- Segment distance and duration values are correctly assigned.
- Geometry data is correctly attached to the appropriate segment.
- Aggregated total distance and duration values are computed accurately.
- The resulting RoutePlan structure remains internally consistent.

Controlled mock routing responses are used to isolate and validate assembly behavior independently from external routing services.

4.1.8 Content Aggregation Tests

Related Classes: *ContentService*, *ContentRepository*, *MediaRepository*, *AudioAssetResolver*

This test group verifies the aggregation and packaging of POI-related content, including textual descriptions, images, and audio assets. Testing ensures that content retrieval remains consistent even when certain assets are unavailable.

Unit tests target the following methods:

- *getPoiContent(String poiId, Language lang)*
- *batchGetContent(List poiIds, Language lang)*

The following aspects are verified:

- Correct retrieval of textual content based on POI identifier and language.

- Proper inclusion of image and audio assets when available.
- Stable behavior when media assets are partially missing.
- Preservation of response structure regardless of asset availability.

Repository dependencies are replaced with controlled mock responses to ensure deterministic verification.

4.1.9 DTO Integrity Tests

Related Classes: *RouteRequest*, *RouteResponse*, *ReplanRequest*, *TripDaySuggestionRequest*, *TripDaySuggestionResponse*, *PoiQuery*, *PoiQueryResponse*, *PoiContentRequest*, *PoiContentResponse*

This test group verifies the structural validity of API-facing request and response payloads. The objective is to ensure that DTOs exchanged between the frontend and backend remain complete, consistent, and serializable across typical workflows.

The following aspects are verified:

- Required fields are present and correctly populated in each DTO type (e.g., city, trip days, selected categories, and constraints in planning requests).
- Response payloads include structurally complete itinerary and route data when a request is successful.
- Warning and error representations preserve expected format and severity information when returned.
- Replanning payloads remain consistent with the provided itinerary snapshot and include only well-formed edit operations (removal, adding, reordering).
- Batch DTOs (e.g., content batch responses) preserve one-to-one mapping between requested identifiers and returned content entries.

These tests validate payload integrity and interface stability independently from business logic correctness.

4.2 Integration Testing

Integration testing in GUIDE verifies the correct interaction between collaborating components after individual unit-level correctness has been established.

While unit tests validate isolated classes using mocks and controlled inputs, integration tests evaluate real implementations working together within architectural boundaries defined in the LLD.

To improve traceability and readability, the integration tests are organized around the package structure defined in the LLD. Every integration test is related to the packages it connects in Figure.1.

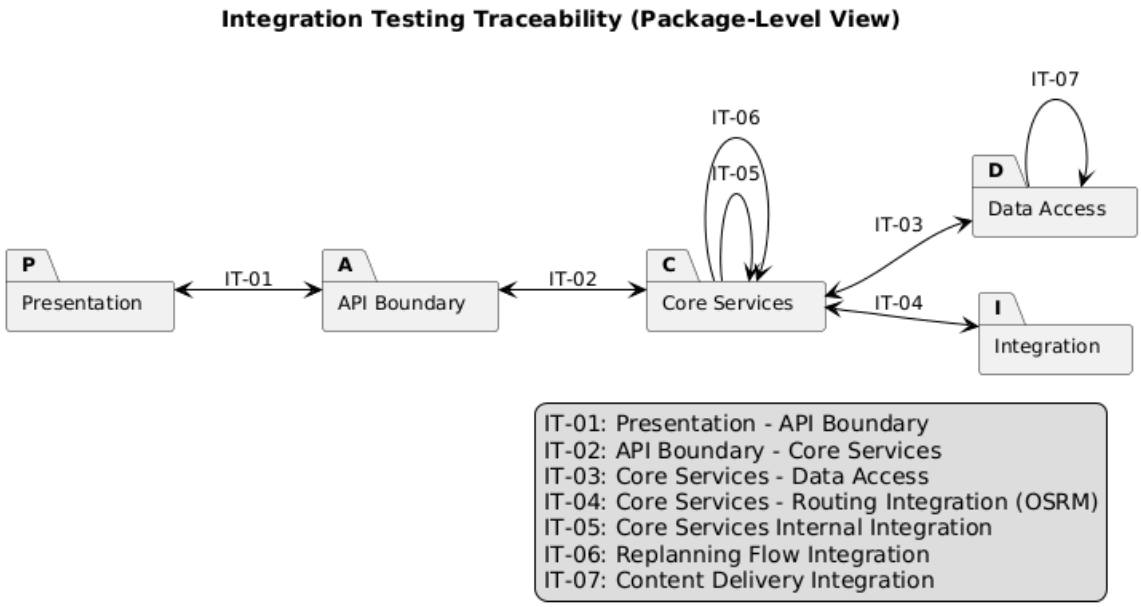

Figure 1: Package-level overview of integration tests.

4.2.1 Presentation – API Boundary Integration

What is Integrated?

This integration validates the contract between the Presentation layer and the backend API boundary. It focuses on how user-provided inputs collected in the frontend are serialized into HTTP requests, mapped into backend DTOs, validated, and returned as normalized success or error responses. The goal is to verify that the frontend and backend interpret the same request/response schema consistently, without testing UI rendering or user experience behavior.

What is Tested?

- Correct serialization and transmission of user inputs from the client to the backend (e.g., *city*, *tripDays*, *categories*, *maxDistancePerDay*, *language*).
- Correct request-to-DTO mapping at the API boundary (JSON payload → DTO fields).
- Validation behavior for required fields, value bounds, and feasibility checks, ensuring that invalid inputs are rejected before reaching core services.
- Correct handling and formatting of error and warning responses returned to the client, including *ApiErrorResponse* and *ApiWarning* structures.
- Correct contract behavior for auxiliary endpoints driven by user input workflows ensuring the response fields are compatible with frontend usage.

What We Aim to Verify

- The frontend request payload format matches the backend DTO structure without field mismatches.

- Unit consistency is preserved across the interface (e.g., km input conversion to meters where applicable).
- Enum and constrained fields (e.g., *Language*) are interpreted consistently across layers.
- Validation outcomes are communicated deterministically and in a frontend-consumable format (clear separation of fatal errors vs non-fatal warnings).
- Responses contain all required fields for subsequent frontend workflows (route visualization, replanning triggers, content retrieval).

Potential Integration Risks

- Field naming mismatches between client payload and backend DTOs (e.g., camelCase inconsistencies).
- Unit mismatches (e.g., frontend sending kilometers while backend expects meters).
- Enum serialization issues (e.g., unsupported language code mapping).
- Validation errors not being mapped into consistent *ApiErrorResponse* payloads.
- Missing response fields causing downstream frontend workflow failures (e.g., warnings omitted, optional fields mis-typed or null unexpectedly).

4.2.2 API Boundary – Core Services Integration

What is Integrated?

This integration connects the API layer with backend business logic. Validated request objects are forwarded from the controller layer to the service layer, and structured responses are constructed and returned.

What is Tested?

- Correct propagation of *RouteRequest* objects
- Proper invocation of *ItineraryService* and *RoutingService*
- Correct handling of *ValidationResult* (errors, warnings, etc.)
- Proper transformation of service-level outputs into *RouteResponse* DTOs

What We Aim to Verify

- No mismatch between DTO structures and service-level domain models
- No data loss between validation and itinerary construction
- Errors are mapped into consistent *ApiErrorResponse* objects
- Warnings are normalized and returned correctly

Potential Integration Risks

- Incorrect field mapping between DTO and domain model
- Service invocation with incomplete constraints
- Improper error propagation
- Missing warning forwarding

4.2.3 Core Services – Data Access Integration

What is Integrated?

Business logic services are executed using the real curated dataset instead of mocked repositories.

What is Tested?

- Correct filtering of POIs by city and category
- Feasibility checks against dataset size
- Retrieval of POI metadata and associated content

What We Aim to Verify

- Repository queries return consistent domain entities
- Filtering logic does not produce empty candidate pools unexpectedly
- Missing content is handled without breaking itinerary construction

Potential Integration Risks

- Category mismatch between dataset and service logic
- Incorrect repository filtering
- Null content propagation
- Dataset integrity inconsistencies

4.2.4 Core Services – Routing Integration (OSRM)

What is Integrated?

The real routing engine is invoked using actual geographic coordinates associated with the POIs selected during itinerary construction.

What is Tested?

- Construction of valid OSRM routing requests from selected POI coordinates
- Correct parsing of OSRM JSON responses
- Proper mapping into *OsrmRouteResponse*

- Accurate assembly of *RoutePlan* and *RouteSegment*

What We Aim to Verify

- Routing outputs remain consistent with the POIs supplied to OSRM
- Geometry encoding is preserved correctly
- Distance and duration values are consistent
- Segment-to-day mapping remains correct
- Timeout and routing failures are handled gracefully

Potential Integration Risks

- Inconsistent POI-to-route mapping
- Geometry parsing errors
- Routing timeout propagation failures

4.2.5 Core Services Internal Integration

What is Integrated?

This integration verifies cooperation between planning, candidate generation, scoring, and the formation of multi-day itinerary structures.

What is Tested?

- Candidate generation and ranking flow
- Structural validity of generated multi-day itinerary candidates
- Deterministic ranking under fixed seed
- Proper aggregation of total distance and duration

What We Aim to Verify

- Ranked itinerary candidates are structurally valid
- No constraint violations within generated itinerary structures
- No inconsistent day indexing
- No duplication or omission of POIs

Potential Integration Risks

- Ranking logic receiving incomplete itinerary objects
- Generation logic producing invalid multi-day itinerary structures
- Distance totals not matching segment-level values
- Constraint leakage between services

4.2.6 Replanning Flow Integration

What is Integrated?

The complete replanning pipeline using the itinerary snapshot provided by the client.

What is Tested?

- Deterministic application of *UserEdits*
- Partial recomputation of affected route segments
- Consistency of updated itinerary totals

What We Aim to Verify

- Edits do not corrupt itinerary structure
- Only necessary route segments are recomputed
- No duplication or missing POIs occur

Potential Integration Risks

- Incorrect day reassignment
- Route recomputation affecting unaffected days
- Lost edits during processing

4.2.7 Content Delivery Integration

What is Integrated?

Complete POI content retrieval pipeline, including media and multilingual audio resolution.

What is Tested?

- Correct retrieval of description text
- Proper image resolution
- Language-based audio selection
- Graceful degradation when assets are missing

What We Aim to Verify

- Content packaging is consistent
- Media references are valid
- Missing audio does not break response structure
- No null pointer propagation to API layer

Potential Integration Risks

- Incorrect language mapping
- Missing media references
- Broken asset path resolution
- Incomplete content object serialization

4.3 System Testing

System testing evaluates the complete end-to-end behavior of the GUIDE system under realistic usage scenarios. Unlike unit and integration testing, which validate isolated components or package-level interactions, system testing verifies the coordinated operation of all architectural layers together.

These tests are designed to simulate real user workflows, including preference-based itinerary generation, routing computation, content exploration, and interactive replanning. All scenarios are executed using the real curated dataset and the locally deployed OSRM routing engine.

The objective is to confirm that the system behaves consistently and predictably across cities with different POI pool sizes, varying constraint configurations, and different types of user interactions.

4.3.1 Scenario 1 – Medium POI Pool Planning

Objective

To evaluate full itinerary generation behavior in a city with a medium POI pool size under typical user constraints.

Initial Conditions

- City: Mersin (69 POIs)
- Trip duration: 2–3 days
- Category selection: 2 main, 5 subcategories
- No daily distance preferences

Execution Steps

1. Submit a route generation request.
2. Observe POI filtering and candidate generation.
3. Review generated daily itinerary structure.
4. Inspect computed routes and geometry on the map.

Expected System Behavior

- A complete multi-day itinerary is generated.

- POIs are distributed without duplication across days.
- Routes are computed successfully via OSRM.
- No structural constraint violations occur.

4.3.2 Scenario 2 – Small POI Pool Handling

Objective

To evaluate system behavior when the available POI pool is limited.

Initial Conditions

- City: Yalova (11 POIs)
- Trip duration: 2 days
- Multiple but not all category selections
- No daily distance preferences

Execution Steps

1. Submit itinerary generation request.
2. Observe POI sufficiency validation.

Expected System Behavior

- The system detects insufficient POI availability.
- Corrective feedback is provided (e.g., system can continue with the present POIs or user can adjust categories).
- No partial or structurally invalid itinerary is produced.

4.3.3 Scenario 3 – Large POI Pool Upper-Bound Planning

Objective

To evaluate system stability and distance-constraint behavior under high candidate workload conditions in a city with a large POI pool.

Initial Conditions

- City: İstanbul (130 POIs)
- Maximum allowed trip duration
- Maximum category selection
- Narrow daily distance preference (e.g., minimum distance for İstanbul)

Execution Steps

1. Submit a route generation request with the specified constraints.
2. Observe selected daily POI distribution.
3. Review computed OSRM route distances and durations.

Expected System Behavior

- The system remains responsive despite the expanded candidate pool.
- Candidate selection reflects sensitivity to the specified daily distance range.
- Generated daily routes remain within or reasonably close to the defined distance bounds.
- Straight-line distance estimates used during scoring remain reasonably aligned with OSRM-computed route distances.
- All itinerary constraints (no duplication, correct day allocation) remain satisfied.

4.3.4 Scenario 4 – Replanning with Structural Edits

Objective

To verify the correct handling of POI removal and reordering operations.

Initial Conditions

- Existing multi-day itinerary generated for Mersin

Execution Steps

1. Remove three POI from a selected day.
2. Reorder two POIs within the same day.
3. Submit replanning request.

Expected System Behavior

- Removed POIs do not reappear.
- Updated order is reflected in route computation.
- Only affected route segments are recomputed.
- No duplication or structural corruption occurs.

4.3.5 Scenario 5 – Replanning with POI Addition

Objective

To evaluate system behavior when new POIs are added to an existing itinerary.

Initial Conditions

- Existing itinerary generated for İstanbul

Execution Steps

1. Add one or more new POIs.
2. Submit replanning request.

Expected System Behavior

- Newly added POIs are incorporated into the itinerary.
- Routes are recomputed accordingly.
- Overall itinerary structure remains consistent.

4.3.6 Scenario 6 – Full End-to-End Workflow Validation

Objective

To validate the complete system behavior from planning to exploration and replanning.

Initial Conditions

- City: İstanbul
- 3 days trip
- Multiple category selection
- No daily distance preferences

Execution Steps

1. Generate itinerary.
2. Explore route visualization.
3. Perform a replanning modification.
4. Open POI content (text, image, audio if available).

Expected System Behavior

- All system components operate cohesively.
- Content delivery aligns with itinerary structure.
- Replanning updates propagate consistently across routing and visualization.
- No inconsistencies occur across layers.

4.3.7 Scenario 7 – Data Source Transparency Accessibility Validation

Objective

To validate that the system provides clear and accessible transparency information regarding routing and content data sources to the user.

Initial Conditions

- Web application is running
- User is on the landing page

Execution Steps

1. Navigate to the "How It Works".
2. Review the displayed information regarding system data sources.

Expected System Behavior

- The transparency page loads successfully without errors.
- The page explicitly states that routing data is based on OpenStreetMap and OSRM [5, 6].
- The page clearly indicates that POI descriptions are sourced from publicly available references (e.g., UNESCO, Wikipedia) [7, 8].
- The information is visibly accessible within the user interface (not hidden or embedded in technical logs).

4.4 Performance Testing

Performance testing in GUIDE evaluates whether the system can satisfy its defined latency requirement under realistic and upper-bound usage scenarios. The primary objective of this evaluation is to verify compliance with the performance requirement.

Unlike large-scale production systems, GUIDE is designed as a bounded-scope academic system operating on a curated dataset within Türkiye. Therefore, performance testing focuses on end-to-end response time behavior and resource stability under representative and worst-case configurations rather than high-concurrency load testing.

4.4.1 Performance Objective

The key performance requirement evaluated in this section is:

- **PR-H01:** Typical 1–3 day itinerary generation should complete within approximately 5 seconds under normal usage conditions.

4.4.2 Latency Evaluation Under Typical Usage

Performance evaluation will first consider a representative "typical user" configuration. In GUIDE, a typical planning scenario is defined as follows:

- Trip duration of 1–3 days,
- Selection of 3–5 POI categories,
- A moderate POI pool size within the selected city.

To provide a quantitative definition of moderate pool size, the full POI dataset will be partitioned into three equal groups based on city-level POI count. Cities in the lowest third (i.e., the bottom 33% in terms of POI availability) will be considered "small-pool" cities. All remaining cities (those not in the lowest third) will be treated as medium or large-pool cities for performance evaluation purposes.

Typical usage scenarios will therefore be executed on cities whose POI count does not fall within the lowest third of the dataset distribution. This ensures that candidate generation and filtering operate on a realistically sized pool rather than a minimal edge-case dataset.

Under these conditions, end-to-end itinerary generation will be measured from the API request reception to the fully assembled route response. This includes:

- Input validation,
- POI filtering and candidate pool formation,
- Monte Carlo candidate generation,
- Scoring and day allocation,
- OSRM route computation,
- Response assembly.

The evaluation will verify whether itinerary generation under typical configurations satisfies the defined 5-second performance requirement. Execution time stability across repeated runs will be examined to identify potential latency fluctuations.

In addition, performance-oriented runs will be checked for structural inconsistencies (such as constraint violations or duplicate POIs across days) to ensure that improvements in speed do not compromise correctness under normal workload conditions.

4.4.3 Upper-Bound Scenario Analysis

To evaluate system robustness under more demanding configurations, upper-bound scenarios will be tested. These represent edge-case configurations approaching defined system constraints.

The following conditions will be examined:

- Maximum allowed trip duration,

- Maximum category selection,
- Large candidate POI pool,
- Increased Monte Carlo candidate generation workload.
- Large candidate POI pool,
- Large POI metadata retrieval workload from the dataset,

Under these configurations, route generation time is expected to increase compared to typical usage due to expanded candidate evaluation and routing workload.

Performance testing will evaluate whether:

- The system remains responsive without excessive latency growth,
- No structural constraint violations occur under high workload,
- Candidate generation and scoring remain stable without exponential time growth,
- POI metadata retrieval latency does not introduce disproportionate delay under large-pool configurations.

Although upper-bound scenarios may approach or exceed typical response times, the evaluation will determine whether the system continues to operate within acceptable academic performance limits without instability, crashes, or inconsistent outputs.

4.4.4 Replanning Performance Considerations

Replanning operations will be evaluated separately due to their partial recomputation nature. Unlike full itinerary generation, replanning does not reconstruct the entire candidate pool from scratch; instead, it operates on the existing itinerary snapshot and applies targeted modifications.

Typical replanning scenarios will include:

- Removal of a limited number of POIs,
- Reordering of POIs within a day,
- Addition of new POIs.

When the number of added or removed POIs remains limited, the computational workload is expected to remain proportional to the size of the affected segments rather than the entire itinerary. Accordingly, replanning latency will be evaluated to determine whether it remains comparable to or lower than full itinerary generation in typical scenarios.

The evaluation will check whether moderate edit operations introduce significant execution time increases or unexpected performance degradation. The system will also be examined to ensure that replanning requests involving standard edits do not trigger unnecessary full recomputation or instability.

4.4.5 Resource Usage Evaluation Plan

In addition to latency evaluation, resource consumption will be monitored during performance testing to ensure stable runtime behavior.

Since GUIDE operates on a curated dataset and a locally deployed OSRM instance, the objective is not large-scale production benchmarking, but verification of stable and bounded resource usage under both typical and upper-bound scenarios.

The following aspects will be examined:

- Backend process memory stability across repeated executions,
- CPU utilization during candidate generation and routing phases,
- OSRM memory usage under repeated route computations.

In addition to planning-related computation, data access and content delivery operations will also be monitored from a resource perspective. This includes retrieval of POI metadata (descriptions, image references) and audio assets during user-triggered interactions (e.g., opening a POI detail view or requesting audio playback). Audio retrieval is expected to be performed on-demand to avoid unnecessary transfer and memory overhead. The evaluation will verify that content retrieval does not introduce disproportionate CPU or memory pressure and does not block the main interaction flow, particularly in cities with large POI pools.

The evaluation will specifically check for:

- Progressive memory growth (potential memory leaks),
- Unbounded CPU spikes under high candidate workloads,
- Resource instability during consecutive replanning operations.

The objective is to confirm that system resource consumption remains bounded and proportional to workload complexity, without introducing runtime instability.

4.5 User Testing

User testing is conducted to collect direct user feedback, measure overall satisfaction, and identify potential areas for improvement in the GUIDE system. While algorithmic components such as Monte Carlo–based itinerary generation and scoring mechanisms can be validated for correctness and constraint compliance, the perceived quality of generated routes cannot be fully evaluated through automated or deterministic tests alone.

Since aspects such as itinerary attractiveness, perceived relevance, and overall user experience involve subjective judgment, user testing serves as a complementary evaluation method. The objective is to assess whether the produced itineraries are considered useful, coherent, and satisfactory by real users under practical usage conditions.

4.5.1 Target Participants

The targeted sample size for user testing is between 40 and 60 participants. Participants will be recruited through voluntary participation within the university community and extended social circles. This sampling approach enables diverse feedback while remaining feasible within the scope of a graduation project.

Participants may access the system from different device types, including desktop computers, laptops, tablets, and mobile phones. This allows usability observations across varying screen sizes and device contexts in alignment with the system's responsive interface design.

At the time of writing this document, user testing has not yet been completed. The final number of participants and a summary of demographic information will be reported after the completion of testing in this subsection.

4.5.2 Test Procedure

Participants will complete a set of predefined task scenarios designed to simulate realistic usage of the GUIDE system. The tasks focus on itinerary generation, route exploration, and replanning functionality. The estimated completion time for the full test is approximately 10 minutes with tasks.

After completing the tasks, participants will submit feedback through a structured Google Form questionnaire that includes Likert-scale evaluation items, task-completion confirmation, and optional open-ended feedback fields.

Google Form Link: `https://forms.gle/3PCuDFwKhmunqYH89`

Participants will perform the test independently on their own devices, and no prior technical knowledge of the system will be required.

4.5.3 User Test Scenarios

Participants will complete three mandatory task scenarios that collectively cover the full end-to-end workflow of the GUIDE system, including itinerary generation, route visualization, and interactive replanning. These tasks evaluate both functional clarity and perceived usability.

Scenario 1 – Preference-Based Itinerary Generation

Participants generate a multi-day itinerary by selecting a destination city, trip duration, preferred daily distance range, and at least one POI category. After generation, they review the structure of the daily plan and assess whether the distribution of POIs across days appears balanced and coherent.

This scenario evaluates clarity of the input process, understandability of the generated itinerary, and perceived relevance of the proposed POIs.

Scenario 2 – Route and Content Exploration

Participants explore the generated itinerary through the map-based route visualization. They inspect at least one daily route segment and open detailed information for at least one POI, including textual descriptions and available media content.

This scenario evaluates the spatial clarity of route presentation, ease of navigation within the interface, and consistency between map visualization and itinerary content.

Scenario 3 – Interactive Replanning

Participants modify the existing itinerary by removing, reordering, or adding at least one POI

and regenerate the updated plan. They then observe how the system adapts to their changes.

This scenario evaluates the intuitiveness of the replanning mechanism, system responsiveness, and perceived consistency of updated routes.

Optional Scenario – Free Exploration

After completing the mandatory tasks, participants may freely explore alternative cities, category combinations, or modified distance ranges. They are encouraged to experiment with different configurations before providing additional feedback regarding their overall experience.

This optional step enables the collection of spontaneous usability insights and subjective evaluations beyond predefined task boundaries.

4.5.4 Evaluation Method and Questionnaire

The evaluation primarily uses a 5-point Likert scale:

- 1 – Strongly Disagree
- 2 – Disagree
- 3 – Neutral
- 4 – Agree
- 5 – Strongly Agree

The questionnaire includes scenario-specific evaluation questions, overall usability assessment, task-completion confirmation for mandatory scenarios, and an optional open-ended feedback item.

4.5.5 Questionnaire Items

Scenario 1 – Itinerary Generation

- The itinerary generation process was clear and easy to follow.
- The generated daily itinerary appeared balanced and logically structured.

Scenario 2 – Route and POI Exploration

- The map-based route visualization was clear and easy to interpret.
- The POI descriptions and content were informative and useful.

Scenario 3 – Replanning

- The updated itinerary reflected my changes correctly.
- The system responded consistently after modifications.

Overall System Evaluation

- The system was easy to use overall.

- The generated itineraries met my expectations.
- I would consider using this system for real trip planning.

Optional Exploration (if completed)

- During free exploration, the system remained stable and predictable.
- The system allowed flexible experimentation without confusion.

Open-Ended Feedback

- Please provide any additional comments or suggestions.

4.5.6 Success Criteria and Analysis Plan

User testing results will be evaluated using predefined quantitative thresholds derived from Likert-scale responses. Since user satisfaction and perceived usability are subjective constructs, measurable success criteria are required to interpret the collected feedback objectively.

For each Likert-scale question, responses will be analyzed using mean score and response distribution. The following criteria define overall success:

- The average score for each mandatory scenario question should be ≥ 3.5 out of 5.
- At least 70% of responses should fall within the Agree" or Strongly Agree" categories (scores 4 or 5) for core usability questions.
- No critical usability issue should be reported repeatedly (more than 20% of participants) in open-ended feedback.

For overall system evaluation:

- The statement "I would consider using this system for real trip planning" should achieve an average score of at least 3.5.
- The overall usability perception should not exhibit extreme polarization (e.g., high variance with large disagreement clusters).

If these thresholds are satisfied, the system will be considered to meet user acceptance expectations within the scope of an academic prototype.

Quantitative analysis will include computation of:

- Mean score per question,
- Standard deviation (to detect polarization),
- Percentage distribution across Likert categories.

Qualitative open-ended responses will be categorized into recurring themes (e.g., usability, clarity, performance, content quality). Repeated themes will be documented and evaluated for potential improvement opportunities.

5 Test Environment

This section describes the hardware and software configurations used during the testing of the GUIDE system. Testing activities are conducted on representative local development environments to ensure cross-platform compatibility and consistent system behavior across different operating systems and hardware architectures.

All system components are executed locally within the described development environments. No external cloud infrastructure, remote servers, or third-party hosted services are used during testing.

The use of standard consumer-grade hardware instead of dedicated server infrastructure demonstrates that GUIDE does not require specialized computational resources and can operate reliably on typical personal computing devices.

Testing is conducted on two representative machines to ensure cross-platform validation. Unless otherwise stated, any **Passed** status reported in this document indicates successful validation on both test environments, ensuring consistent behavior across the defined hardware and software configurations.

5.1 Hardware

Primary Test Machine (Windows Platform)

- Operating System: Microsoft Windows 11 Home Single Language (10.0.26200)
- CPU: 11th Gen Intel® Core™ i7-11800H @ 2.30GHz
- Memory: 32 GB RAM
- Storage: 2 TB NVMe SSD (Samsung MZVL22T0HBLB-00B00), approximately 1012 GB free
- GPU: NVIDIA GeForce RTX 3070 Laptop GPU
- Network Setup: Local execution environment

Secondary Test Machine (macOS ARM Platform)

- Operating System: macOS 26.3 (ARM64 architecture)
- CPU: Apple M1 (8-core)
- Memory: 8 GB RAM
- Storage: 186 GB SSD (local disk)
- GPU: Integrated Apple M1 GPU (8-core)
- Network Setup: Local execution environment

5.2 Software

Primary Test Machine (Windows Platform)

- Git: 2.52.0.windows.1
- Python: 3.13.11
- Node.js: v22.16.0
- Database: PostgreSQL 17

Secondary Test Machine (macOS ARM Platform)

- Git: 2.50.1
- Python: 3.12.7
- Node.js: v22.22.0
- Database: PostgreSQL 16.12

6 Test Cases

The test cases listed in Table 1 summarize the planned validation activities across unit, integration, system, performance, and user testing levels. The **Result** field is left blank until execution is completed. Once a test case is executed, its outcome will be recorded as **Passed** or **Failed** accordingly.

Table 1: Test cases for the GUIDE system.

Test ID	Description	Input(s)	Expected Output	Result
Unit Testing (UT)				
TC-UT-01	Reject request when *city* is missing	RouteRequest: *city=null*, other fields valid	*ValidationResult.valid=false*; error indicates missing *city*.	**Passed**
TC-UT-02	Reject request when *categories* is empty/missing	RouteRequest: *categories=[]*, other fields valid	*ValidationResult.valid=false*; error indicates missing/empty *categories*.	**Passed**
TC-UT-03	Require at least one planning constraint (days or distance)	RouteRequest: *tripDays=null* and *maxDistancePerDay=null* (other fields valid)	*ValidationResult.valid=false*; error indicates *tripDays* or *maxDistancePerDay* must be provided.	**Passed**
TC-UT-04	RouteRequest enforces *maxTripDays* upper bound	RouteRequest with *tripDays = maxTripDays+1*	*ValidationResult.valid=false* with a bound-violation error for *tripDays*.	**Passed**
TC-UT-05	RouteRequest enforces a city-based upper bound for daily distance	RouteRequest with *city=X* and *maxDistancePerDay > maxAllowedDistance(city=X)*	*ValidationResult.valid=false* with a bound-violation error for *maxDistancePerDay* (exceeds city-based limit).	**Passed**

Test ID	Description	Input(s)	Expected Output	Result
TC-UT-06	POI filtering returns only selected city	Prefs: city=İstanbul, categories=[...]	All returned POIs satisfy *poi.city == İstanbul*	**Passed**
TC-UT-07	POI filtering returns only selected categories	Prefs: city=İstanbul, categories=[Museum, Park]	All returned POIs satisfy *poi.category* $\in$ *Museum, Park*	**Passed**
TC-UT-08	Candidate generation produces no duplicates within a candidate	Fixed seed; POI list size > 0	Each generated candidate itinerary contains unique POI IDs	
TC-UT-09	Score function is deterministic for identical inputs	Same itinerary + prefs + constraints twice	*score(run1) == score(run2)*	
TC-UT-10	Best-candidate selection always picks the maximum-scoring candidate	A fixed candidate list with known scores (including a unique max score)	The selected itinerary is the one with $\max$ score among candidates (no lower-scoring candidate is chosen).	
TC-UT-11	System returns insufficiency warning after partial generation	Insufficient eligible POI pool relative to the requested number of days	Response includes a non-fatal warning indicating insufficient POIs and suggesting category adjustment or acceptance of the current route.	
TC-UT-12	No POI is assigned to more than one day	Allocated itinerary for tripDays=3	Set of POI IDs across all days has no duplicates	
TC-UT-13	OSRM response parsing extracts distance, duration, and geometry fields	Mock OSRM JSON with *distance*, *duration*, and *geometry*	Parsed response has non-null *distanceMeters*, *durationSeconds*, and non-empty *geometry*.	
TC-UT-14	Post-routing output satisfies all request constraints (final gate)	RouteRequest constraints (city, categories, tripDays, distance preference) + generated itinerary + OSRM route response	After OSRM routing, the returned route plan is re-validated and satisfies all constraints: (i) *route.days.size() == requestedTripDays*, (ii) all POIs belong to the requested *city*, (iii) all POIs belong to the selected *categories*, (iv) daily routed distances comply with the declared distance policy (within defined tolerance), and (v) the route structure remains well-formed (no missing segments/invalid day indexing).	
TC-UT-15	RouteAssembler preserves segment-to-day mapping correctness	Itinerary with N days + mocked OSRM responses ordered by day	Each *RouteSegment* is attached to the correct itinerary day; no day receives another day's geometry	

Test ID	Description	Input(s)	Expected Output	Result
TC-UT-16	RoutePlan totals equal the sum of per-segment values	RoutePlan with segments containing distances/durations	*totalDistance == sum(segmentDistances)* and *totalDuration == sum(segmentDurations)* (within rounding tolerance)	
TC-UT-17	Replanning applies remove edit correctly	Existing itinerary + *UserEdits.remove=[poiA, poiB]*	Removed POI IDs are absent from all days; itinerary remains structurally valid	
TC-UT-18	Replanning applies reorder edit correctly within a day	Existing itinerary day with POIs [p1,p2,p3] + reorder request to [p2,p1,p3]	Updated itinerary day order matches the requested order exactly	
TC-UT-19	Replanning applies add edit correctly	Existing itinerary + add request with new POI(s) + insertion target (day/position)	Added POI IDs appear exactly once at the requested location; no duplication across days	
TC-UT-20	Replanning does not corrupt unaffected days	Multi-day itinerary + edits affecting only Day k	Days other than k remain identical (same POI IDs and ordering) before vs after replanning	
TC-UT-21	Replanning recomputes routes only for affected day/segments	Existing RoutePlan + edits affecting Day k + routing call tracker/mock	Routing service is invoked only for affected day/segments; unaffected days are not recomputed	
TC-UT-22	Replanning preserves constraint compliance (no duplicates, valid day indexing)	Existing itinerary + mixed edits (remove+reorder+add)	The resulting itinerary has no duplicate POI IDs across days and maintains a valid day structure/indexing	
TC-UT-23	Replanning rejects invalid edit referencing non-existent POI	Existing itinerary + edit referencing POI ID not in itinerary/dataset	Validation fails with deterministic error; no partial update is produced	
TC-UT-24	Content delivery returns text description	POI content request for POI with known description	Response contains non-empty *descriptionText*	
TC-UT-25	Content delivery returns image reference(s) when available	POI content request for POI with image URL/path(s)	Response contains valid image reference field(s) (non-empty) without serialization errors	
TC-UT-26	Audio content retrieval succeeds for supported languages when available	POI content request with *lang=TR/EN/DE* where audio exists	Response includes an audio reference/asset for the requested language (or defined fallback policy)	
TC-UT-27	Missing image is handled gracefully without breaking the response	POI content request where image is missing/null	Response still includes text fields; image field omitted/marked missing; no exception is thrown	

Test ID	Description	Input(s)	Expected Output	Result
TC-UT-28	Missing audio is handled gracefully without breaking the response	POI content request where audio is missing for the requested language	Response still includes text/image; audio omitted/marked unavailable; no exception is thrown	
TC-UT-29	Batch content retrieval preserves one-to-one mapping	Batch request with POI IDs [a,b,c]	Batch response contains exactly 3 entries; each entry maps correctly to its POI ID (order preserved)	
Integration Testing (IT)				
TC-IT-01	Frontend → API: request schema maps to DTO fields without mismatch	HTTP JSON with *city*, *tripDays* or *maxDistancePerDay*, *categories*, *language*	Backend DTO fields populated correctly; no unintended nulls from mapping; request accepted and forwarded	
TC-IT-02	Frontend → API: unit consistency across boundary (km → internal units)	Client sends *maxDistancePerDay=10* (km) (UI uses km)	Backend constraint matches expected internal representation (e.g., meters)	
TC-IT-03	Frontend → API: error payload contract is stable and parseable	Invalid client request (e.g., missing *city*)	HTTP 4xx; response matches *ApiErrorResponse* schema (message, code/fields if defined); frontend can parse without fallback errors	
TC-IT-04	Frontend → API: warning payload contract is stable (non-fatal guidance)	Request that triggers non-fatal warning (e.g., insufficient POIs but system provides partial route + guidance)	HTTP 200/2xx (per design) with *ApiWarning* included; warning is distinguishable from fatal error and includes guidance text/fields	
TC-IT-05	API Boundary → Core: controller orchestrates correct service invocation chain	Valid route generation request	*ItineraryService* and *RoutingService* invoked in expected order; response assembled into *RouteResponse* without missing fields	
TC-IT-06	API Boundary → Core: validation failure blocks core service execution	Invalid request that fails validation	Core services are not invoked; API returns consistent *ApiErrorResponse* (or designed failure response) without partial computation side-effects	
TC-IT-07	API Boundary → Core: warning propagation is preserved end-to-end	Request that yields a warning in core (e.g., partial generation/content missing policy)	Warnings produced in core appear in API response without loss; severity/type preserved; response remains usable for UI workflows	
TC-IT-08	Core → Data: real dataset retrieval returns consistent domain entities (no contract break)	City with known POIs (e.g., İstanbul) + category subset	Repository returns POIs with required fields (IDs, coords, category); core pipeline proceeds without null-pointer/contract errors	

Test ID	Description	Input(s)	Expected Output	Result
TC-IT-09	Core → Data: category mapping mismatch is detectable (no silent empty-pool failure)	Use a category value that risks mapping mismatch (or simulate mismatched mapping)	System returns deterministic error/warning indicating category mismatch or no matching POIs (per design)	
TC-IT-10	Core → Data: partial/missing content does not break planning pipeline	City/categories where some POIs lack optional content (image/audio)	No crash in route generation due to missing optional assets	
TC-IT-11	Core → OSRM: geometry + distance + duration are returned and serialized for map rendering	Real OSRM calls for a generated itinerary	Route response includes non-empty geometry/polyline plus distance and duration fields in a consistent format required by frontend map rendering	
TC-IT-12	Core → OSRM: routing timeout/failure handled gracefully without breaking API contract	Induce OSRM failure (timeout/unreachable) during route computation	API returns deterministic failure response (error schema stable); system does not hang indefinitely; no partially corrupted response returned	
TC-IT-13	Replanning pipeline: client itinerary snapshot + edits are applied and returned consistently	Existing itinerary snapshot + mixed edits (remove/reorder/add in one request)	Updated itinerary returned; edits reflected correctly; response schema stable for UI; no duplication/loss beyond intended edits	
TC-IT-14	Replanning pipeline: only affected day/segments trigger recomputation (partial recompute)	Multi-day itinerary + edits affecting a single day	Routing/assembly recomputed only for affected day/segments	
TC-IT-15	Content delivery: endpoint response structure remains stable with missing media	POI content request where image/audio may be missing	Response still contains required text fields; missing media is omitted or marked unavailable; serialization succeeds; UI can render gracefully	
TC-IT-16	Content delivery: language-based audio selection/fallback works end-to-end	POI content request with *lang=TR* and *lang=EN* and *lang=DE* (where availability differs)	If asset exists: correct language audio reference returned. If not: defined fallback policy applied deterministically without breaking schema	
TC-IT-17	Content delivery: batch retrieval preserves mapping between requested IDs and returned entries	Batch request with POI IDs $[a, b, c]$	Batch response contains exactly 3 entries and each entry maps to the correct POI ID (order preserved if specified); no dropped/duplicated entries	

Test ID	Description	Input(s)	Expected Output	Result
System Testing (ST)				
TC-ST-01	Scenario 1: End-to-end planning produces a usable multi-day route plan	Mersin (69); tripDays=3; categories=3–5; no distance preference	A full end-to-end response is produced and is consumable by the UI workflow: The itinerary is present for the requested days, and a route plan is returned for map rendering.	
TC-ST-02	Scenario 2: End-to-end flow returns user-facing insufficiency guidance without breaking the workflow	Yalova (11); tripDays=2; limited categories; no distance preferences	System responds with a user-facing insufficiency warning message that guides the user to adjust categories or accept a reduced plan.	
TC-ST-03	Scenario 3: Distance preference is reflected in the final routed output end-to-end	İstanbul (137); near-maximum tripDays; maximum categories; narrow daily distance target (e.g., 10 km)	End-to-end output reflects distance-sensitive behavior at the routed level: daily OSRM route distances are within/near the target range according to the defined tolerance policy, and the system completes without instability.	
TC-ST-04	Scenario 4: Replanning updates propagate end-to-end to routing + visualization	Existing Mersin plan; remove 3 POIs from Day k; reorder 2 POIs in Day k; submit replan	Updated itinerary and route plan are returned successfully; the client can visualize the updated route without errors, and changes are reflected in the final day route output.	
TC-ST-05	Scenario 5: Adding POIs via replanning works end-to-end without breaking other days	Existing İstanbul plan; add 1–N POIs to a chosen day/position; submit replan	Replanned response is produced successfully and remains UI-consumable; added POIs appear in the updated day route visualization, and the workflow remains stable (no unintended disruption of unrelated days visible to the user).	
TC-ST-06	Scenario 6: Full workflow (plan → explore → content → replan) completes cohesively	İstanbul; tripDays=3–5; multiple categories; perform one replan edit; open POI detail (text/image/audio if available)	All user-visible steps complete without client-visible errors: route renders, a replanning action yields an updated route that remains consistent in the UI, and POI detail loads (or gracefully indicates missing assets).	

Test ID	Description	Input(s)	Expected Output	Result
TC-ST-07	Scenario 7: Data source transparency page is accessible, and displays required attribution information	Application running; user starts from landing page; navigate to "How It Works" (or equivalent transparency) page	Transparency page loads without errors; routing data attribution (OpenStreetMap/OSRM) and POI content source attribution (public references such as UNESCO/Wikipedia) are clearly displayed and readable within the UI	
Performance Testing (PT)				
TC-PT-01	Typical usage latency satisfies PR-H01 ($\leq$ 5 seconds)	1–3 days; 3–5 categories; medium/large pool city	Average end-to-end latency (multiple runs) $\leq$ 5 seconds	
TC-PT-02	Upper-bound configuration does not cause instability	Max tripDays; max categories; large pool city	System completes without crash or timeout; latency growth remains bounded	
TC-PT-03	Large metadata retrieval does not introduce disproportionate delay	Large pool city; content metadata retrieval triggered	Content retrieval latency does not dominate total response time	
TC-PT-04	Replanning latency remains comparable to full planning	Typical itinerary; limited remove/reorder/add edits	Replanning latency $\leq$ full planning latency under similar conditions	
TC-PT-05	Backend memory usage remains stable across repeated executions	Repeated planning + replanning cycles	No progressive memory growth indicative of leak; memory plateaus	
TC-PT-06	CPU usage remains bounded under high candidate workload	Upper-bound planning scenario	No sustained abnormal CPU spike; usage proportional to workload	
TC-PT-07	On-demand audio retrieval does not block the main interaction flow	Open POI detail; trigger audio playback	Audio retrieved on-demand; no UI freeze; main route flow unaffected	
User Testing (UST)				
TC-UST-01	Scenario 1 mean score threshold satisfied	Likert responses	Mean score $\geq$ 3.5	
TC-UST-02	Scenario 1 agreement ratio threshold satisfied	Likert responses	$\geq$ 70% of responses are 4 or 5	
TC-UST-03	Scenario 2 mean score threshold satisfied	Likert responses	Mean score $\geq$ 3.5	
TC-UST-04	Scenario 2 agreement ratio threshold satisfied	Likert responses	$\geq$ 70% of responses are 4 or 5	
TC-UST-05	Scenario 3 mean score threshold satisfied	Likert responses	Mean score $\geq$ 3.5	
TC-UST-06	Scenario 3 agreement ratio threshold satisfied	Likert responses	$\geq$ 70% of responses are 4 or 5	
TC-UST-07	Overall usability threshold satisfied	Likert responses	Mean score $\geq$ 3.5	

Test ID	Description	Input(s)	Expected Output	Result
TC-UST-08	Overall usability agreement ratio threshold satisfied	Likert responses	$\geq$ 70% of responses are 4 or 5	
TC-UST-09	Real-world usage intention threshold satisfied	"I would consider using this system…" item	Mean score $\geq$ 3.5	
TC-UST-10	Real-world usage intention agreement ratio threshold satisfied	"I would consider using this system…" item	$\geq$ 70% of responses are 4 or 5	
TC-UST-11	No critical recurring usability issue	Open-ended responses analysis	No single critical issue reported by $>$ 20% of participants	
TC-UST-12	No extreme polarization in usability perception	Variance / standard deviation analysis	No extreme variance indicating major disagreement clusters	

In total, the test suite defines 72 test cases across unit (29), integration (17), system (7), performance (7), and user testing (12).

7 Risks and Limitations of the Test Strategy

This section outlines limitations related specifically to the testing scope and evaluation methodology adopted in this document. While GUIDE has been validated across multiple testing levels (unit, integration, system, performance, and user testing), certain aspects remain outside the direct verification boundaries of this test plan.

7.1 Heuristic Optimality Not Formally Verified

The testing strategy validates determinism, constraint compliance, and ranking correctness within generated candidate sets. However, it does not formally verify the global optimality of the produced itineraries.

Since the planning algorithm is heuristic-based, the test suite ensures internal consistency and correct maximum-score selection among the generated candidates, but it does not prove that the selected solution is globally optimal within the full combinatorial space. In addition, limited user-based evaluation activities were conducted to collect qualitative feedback regarding itinerary relevance and perceived route quality. However, such evaluations are inherently subjective and context-dependent.

7.2 Dataset Quality Not Exhaustively Audited

Tests validate correct filtering, category consistency, and structural integrity of retrieved POIs. However, the semantic correctness, completeness, or real-world accuracy of the curated dataset are not exhaustively verified through automated testing.

The curated dataset has undergone internal peer review and manual validation within the project team during its preparation phase. However, the test plan does not perform exhaustive semantic auditing of every individual POI entry, and large-scale manual re-verification of all

content is outside the scope of this document.

7.3 External Routing Accuracy Outside Test Scope

Integration tests verify correct invocation of the routing engine, response parsing, geometry preservation, and failure handling. However, the geographic correctness or real-world optimality of OSRM-produced routes is not independently audited.

Routing accuracy is therefore treated as an external dependency, and the test suite focuses on contract correctness rather than geographic validation.

7.4 Replanning Preference Deviation

The test suite verifies structural correctness during replanning, including edit application, duplication prevention, and partial recomputation consistency.

However, after user-driven modifications, the system does not reapply full constraint validation. Tests, therefore, do not enforce strict adherence to original distance or workload preferences following edits.

This behavior reflects a deliberate architectural decision aimed at balancing computational feasibility and user autonomy during iterative planning. Enforcing full global constraint re-evaluation after each user modification would require repeated end-to-end re-optimization, introducing additional computational overhead and potentially degrading interactive responsiveness.

Moreover, user-driven edits are treated as intentional overrides rather than optimization inputs. In practical trip planning scenarios, users may consciously relax earlier preferences (e.g., adding more POIs despite distance targets). Automatically rejecting or correcting such edits could introduce unnecessary friction and reduce usability.

7.5 Performance Under Extreme Concurrency

Performance validation within this test plan focuses on representative and upper-bound single-user workloads that reflect the intended academic deployment context of GUIDE. Latency, memory stability, and routing behavior are evaluated under realistic planning and replanning scenarios.

However, large-scale concurrent stress testing (e.g., dozens or hundreds of simultaneous users issuing route generation requests) is outside the scope of this evaluation. Such testing would require additional infrastructure setup, load-generation tooling, and deployment configurations beyond the academic prototype environment.

7.6 Subjective Evaluation Variability

User testing introduces quantitative evaluation criteria, including mean-score thresholds and agreement ratios, to provide structured measurement of user satisfaction. These metrics allow statistical interpretation of perceived usability and itinerary coherence.

Nevertheless, itinerary attractiveness and perceived relevance are inherently subjective constructs influenced by individual travel preferences, cultural familiarity, prior knowledge of destinations, and personal expectations.

While the test plan establishes measurable acceptance boundaries, it does not attempt to universalize subjective quality perception. Variability across participants is expected and treated as a natural characteristic of experiential evaluation rather than a structural system defect.

8 Conclusion

This test plan defines a structured verification strategy for GUIDE across unit, integration, system, performance, and user testing levels. In total, the suite specifies 74 test cases and is expected to provide approximately 65–75% overall test coverage of the implemented, in-scope functionalities and critical cross-layer contracts.

By combining deterministic unit validation, package-level integration checks, end-to-end scenario execution, performance measurements, and user acceptance criteria, the plan provides a repeatable and traceable basis for confirming system correctness and reliability before final delivery.

A Requirement Traceability Matrix

Table A establishes traceability between the requirements defined in the Project Specification Report and the test cases defined in Chapter 6 of this Test Plan [1].

Each requirement is mapped to the corresponding validation activities (unit, integration, system, performance, or user tests) that verifies its correct implementation.

Requirement ID(s)	Short Description	Related Test Cases
FR-H01	User input provision and validation for route generation.	TC-UT-01–05, TC-IT-01-02
FR-H02	Day-by-day route generation aligned with user constraints.	TC-UT-06–16, TC-IT-05, TC-IT-08-10, TC-IT-18, TC-ST-01, TC-ST-03, TC-ST-06
FR-H03	User-driven modification (remove/reorder/add) and route regeneration.	TC-UT-12–23, TC-IT-13-14, TC-ST-04-05
FR-H04	Display of destination descriptions, images, and map location.	TC-UT-24–25, TC-UT-27, TC-UT-29, TC-IT-15
FR-H05	Multilingual text-to-speech support (TR/EN/DE).	TC-UT-26, TC-UT-28, TC-UT-29, TC-IT-16
FR-H06	Visualization of generated routes on an interactive map.	TC-UT-13, TC-IT-11
FR-H07	Visit duration estimation is incorporated into itinerary planning.	TC-UT-04, TC-UT-14,

Requirement ID(s)	Short Description	Related Test Cases
FR-H08	Graceful error handling and prevention of invalid-input crashes.	TC-UT-01–05, TC-UT-11, TC-UT-27-28, TC-IT-01, TC-IT-03-04, TC-IT-06-07, TC-IT-12,TC-ST-02
FR-H09	Transparency of routing and content data sources.	TC-ST-07
EIR-H01	Web-based, responsive, and intuitive user interface.	TC-UST-01-12
PR-H01	Average route generation latency 5 seconds.	TC-PT-01-07

Note: Spell checking and basic grammatical verification were performed using Grammarly: (`https://www.grammarly.com`).

References

[1] GUIDE Project Team, "Project specification report," TOBB ETU Computer Engineering Department, Tech. Rep., 2026.

[2] ——, "High-level design document," TOBB ETU Computer Engineering Department, Tech. Rep., 2026.

[3] ——, "Low-level design document," TOBB ETU Computer Engineering Department, Tech. Rep., 2026.

[4] "pytest: helps you write better programs," https://docs.pytest.org/, accessed: 2026.

[5] OpenStreetMap Foundation. (2023) OpenStreetMap. [Online]. Available: https://www.openstreetmap.org

[6] OSRM Project. (2023) Open Source Routing Machine HTTP API Documentation. [Online]. Available: http://project-osrm.org

[7] UNESCO. UNESCO World Heritage Centre – Türkiye. Access Date: 2025-09-16. [Online]. Available: https://www.unesco.org.tr

[8] Wikimedia Foundation. (2023) MediaWiki REST API Documentation. [Online]. Available: https://www.mediawiki.org/wiki/API:REST_API